

Angular Learning Series

Topics 33 – 37

Complete Topic Reference — All 16 Sections Per Topic

Generated: 17 May 2026

Topic 33 — Reactive Forms (FormGroup, FormControl, FormArray)

Topic 34 — Form Validation (Built-in, Custom, Async Validators)

Topic 35 — Template-Driven Forms

Topic 36 — HttpClient & API Integration

Topic 37 — HTTP Interceptors

TOPIC 33 — Reactive Forms (FormGroup, FormControl, FormArray)

Angular Core — Forms | Difficulty: Intermediate | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — Reactive Forms introduce a programmatic, model-driven approach that requires understanding class composition, observables, and validation trees working in concert — more moving parts than most Angular topics to this point.

Angular Relevance: Critical — Reactive Forms are the standard for any non-trivial form in Angular. Login screens, multi-step wizards, dynamic field arrays, real-time validation — all of these live here. If you build Angular apps professionally, you will write Reactive Forms every day.

Section 1 — Explanation

The Blueprint Analogy

Imagine you're building a house. You have two approaches:

Approach A (Template-Driven): You tell the workers verbally what to build as they go. Simple shed? Fine. But if the structure is complex, verbal instructions get messy fast — you can't easily inspect the plan, test it, or change it programmatically.

Approach B (Reactive/Blueprint-Driven): You draw a precise blueprint before construction. Every room (FormGroup), every fixture (FormControl), every repeated unit like identical apartments (FormArray) is defined explicitly in the blueprint. You can inspect it, validate it, change it programmatically, and test it without ever touching the physical building.

Reactive Forms are the Blueprint Approach. The form model lives entirely in your TypeScript class — not in the template. The template merely connects to the model. This separation is what makes Reactive Forms testable, scalable, and predictable.

Technical Explanation

Reactive Forms are built from three core building blocks:

Building Block	Role	Analogy
FormControl	A single field	One input box (one room fixture)
FormGroup	A collection of controls	A group of related fields (one floor of the house)
FormArray	A dynamic list of controls or groups	Identical repeated apartments on a floor
AbstractControl	Base class for all three	The shared blueprint standard all three follow

The entire form is a tree of these three classes. At runtime, every control tracks its current value, validity status (valid, invalid, pending), dirty/pristine status, touched/untouched status, a valueChanges observable, and a statusChanges observable.

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 13 — TypeScript Interfaces & Types: You'll type your form values with interfaces
- Topic 16 — TypeScript Generics: FormControl<string>, FormGroup<{...}> use generics for strict typing
- Topic 20 — RxJS Subscription Management: valueChanges and statusChanges are Observables
- Topic 21 — takeUntilDestroyed & DestroyRef: Used to auto-unsubscribe from form observables
- Topic 23 — Angular Signals: Signals and Reactive Forms can be bridged with toSignal(form.valueChanges)
- Topic 24 — OnPush Change Detection: Reactive Forms work cleanly with OnPush because the model is explicit
- Topic 27 — Services & Dependency Injection: FormBuilder is an injectable service

Bridging note: Topics 20 and 21 taught you that observables need cleanup. A

FormControl.valueChanges subscription is no different — if you subscribe manually inside a component, you must unsubscribe. This is the most common memory leak in Angular form code.

Extends most directly: Topic 20 — RxJS Subscription Management, because the entire reactive power of forms flows through observable streams.

Section 3 — Code Example

Plain TS — Core concept in isolation

```
import { FormControl, FormGroup, FormArray, Validators } from '@angular/forms';

// --- FormControl: one field ---
const emailControl = new FormControl('', [Validators.required, Validators.email]);
console.log(emailControl.value); // ''
console.log(emailControl.valid); // false

emailControl.setValue('user@example.com');
console.log(emailControl.valid); // true

// --- FormGroup: related fields grouped together ---
const loginForm = new FormGroup({
  email: new FormControl('', [Validators.required, Validators.email]),
  password: new FormControl('', [Validators.required, Validators.minLength(8)])
});

loginForm.patchValue({ email: 'user@example.com' });

// --- FormArray: dynamic list of controls ---
const tagsArray = new FormArray([
  new FormControl('angular'),
  new FormControl('typescript')
]);
tagsArray.push(new FormControl('rxjs'));
tagsArray.removeAt(0);
console.log(tagsArray.value); // ['typescript', 'rxjs']
```

Real Angular Component Example

```

@Component({
  selector: 'app-profile-form',
  standalone: true,
  imports: [ReactiveFormsModule, CommonModule],
  template: `
    <form [formGroup]="profileForm" (ngSubmit)="onSubmit()">
      <input formControlName="username" />
      @if (username.invalid && username.touched) {
        <span class="error">Username is required</span>
      }
      <div formArrayName="skills">
        @for (skill of skills.controls; track $index) {
          <input [formControlName]="$index" />
          <button type="button" (click)="removeSkill($index)">Remove</button>
        }
      </div>
      <button type="button" (click)="addSkill()">+ Add Skill</button>
      <button type="submit" [disabled]="profileForm.invalid">Save</button>
    </form>
  `
})
export class ProfileFormComponent {
  private fb = inject(FormBuilder);

  profileForm: FormGroup = this.fb.group({
    username: ['', [Validators.required, Validators.minLength(3)]],
    email: ['', [Validators.required, Validators.email]],
    skills: this.fb.array([this.fb.control('')])
  });

  constructor() {
    // takeUntilDestroyed must be in injection context
    this.profileForm.valueChanges
      .pipe(takeUntilDestroyed())
      .subscribe(value => console.log('Form changed:', value));
  }

  get username() { return this.profileForm.get('username')!; }
  get skills() { return this.profileForm.get('skills') as FormArray; }

  addSkill() { this.skills.push(this.fb.control('', Validators.required)); }
  removeSkill(i: number) { this.skills.removeAt(i); }
  onSubmit() { if (this.profileForm.valid) console.log(this.profileForm.value); }
}

```

Section 4 — Before & After Refactor

Before — The Naive Way

```
// WRONG: No validators, unmanaged subscription, typed as 'any'
export class OldFormComponent implements OnInit, OnDestroy {
  loginForm = new FormGroup({
    email: new FormControl(''), // no validators – always 'valid'
    password: new FormControl('')
  });
  private sub: any; // loses type safety

  ngOnInit() {
    this.sub = this.loginForm.valueChanges.subscribe(val => console.log(val));
    // no cleanup – memory leak guaranteed
  }

  onSubmit() {
    console.log(this.loginForm.value); // no validity check – dangerous
  }

  ngOnDestroy() { this.sub.unsubscribe(); } // easy to forget
}
```

After — The Correct Way

```
export class NewFormComponent {
  private fb = inject(FormBuilder);

  loginForm = this.fb.group({
    email: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(8)]]
  });

  constructor() {
    this.loginForm.valueChanges
      .pipe(debounceTime(300), takeUntilDestroyed())
      .subscribe(val => console.log(val));
  }

  onSubmit() {
    if (this.loginForm.invalid) return;
    console.log(this.loginForm.value);
  }
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Forgetting to import ReactiveFormsModule

What: Template throws 'Can't bind to formControlName since it isn't a known property'. Why developers make it: They add FormGroup to the TS class but forget the template directives live in ReactiveFormsModule. How to avoid: Always add ReactiveFormsModule to standalone component imports. Angular-specific consequence: The form silently doesn't bind — inputs are uncontrolled, values never captured.

Mistake 2 — Calling takeUntilDestroyed() outside injection context

What: Placing takeUntilDestroyed() inside ngOnInit() instead of the constructor. Why: ngOnInit feels like the right setup place. How to avoid: Subscribe in the constructor or field initializer. Angular-specific consequence: Runtime error: 'takeUntilDestroyed() can only be used within an injection context.'

Mistake 3 — Using setValue when patchValue is needed

What: Calling setValue({ email: 'x@x.com' }) on a FormGroup that has more fields. Why: Both methods look similar. How to avoid: setValue requires every control; patchValue accepts partial. Angular-specific consequence: setValue throws at runtime with 'Must supply a value for form control with name: password'.

Section 6 — Do's and Don'ts

DO	DON'T
Use FormBuilder (inject(FormBuilder)) to reduce boilerplate	Manually new FormControl() / new FormGroup() in every component
Subscribe to valueChanges in the constructor with takeUntilDestroyed()	Subscribe in ngOnInit without cleanup — guaranteed memory leak
Use patchValue when pre-filling from API data (partial updates)	Use setValue for partial updates — it throws if any control is missing
Add a validity guard (if (form.invalid) return) before onSubmit	Trust [disabled] alone — it can be bypassed via developer tools
Use getter accessors for cleaner templates	Repeat profileForm.get('email') inline in the template multiple times
Type your form value with an interface	Use any for form values — defeats TypeScript's purpose

Section 7 — Debugging Tips

Bug 1 — formControlName must be used with a parent formGroup directive

Symptom: Error 'formControlName must be used with a parent formGroup directive'. Cause: Missing [formGroup]="myForm" on the parent form element. Fix: Add [formGroup]="loginForm" to the form element. Angular note: Also triggered when nesting FormGroups without formGroupName on the wrapper.

Bug 2 — FormArray inputs not binding correctly

Symptom: Inputs in a @for loop don't update the model. Cause: The formArrayName directive is missing on the container, or [formControlName]="\$index" uses wrong syntax. Fix: Add formArrayName="skills" on the container div and use [formControlName]="\$index" (bound, not static).

Bug 3 — reset() leaves the form in touched state

Symptom: After form.reset(), validation error messages still appear. Cause: reset() clears values and dirty but not touched state by default. Fix: Call form.reset() then form.markAsUntouched() then form.markAsPristine().

Section 8 — Real-World Applications

Scenario 1 — User Registration Form with Live Validation

A registration page with username, email, password, confirmPassword. All validation errors shown inline only after the user touches each field. Uses a group-level cross-field validator for password matching.

```

registerForm = this.fb.group(
  {
    username:      ['', [Validators.required, Validators.minLength(3)]],
    email:         ['', [Validators.required, Validators.email]],
    password:      ['', [Validators.required, Validators.minLength(8)]],
    confirmPassword: ['', Validators.required]
  },
  { validators: this.passwordMatchValidator }
);

passwordMatchValidator(group: AbstractControl) {
  const pass      = group.get('password')?.value;
  const confirm   = group.get('confirmPassword')?.value;
  return pass === confirm ? null : { passwordMismatch: true };
}

```

Scenario 2 — Dynamic Invoice Line Items with FormArray

An invoice builder lets users add/remove line items (description, quantity, price), computing a running total reactively.

```

invoiceForm = this.fb.group({
  lineItems: this.fb.array([this.createItem()])
});

get lineItems() { return this.invoiceForm.get('lineItems') as FormArray; }

get total(): number {
  return this.lineItems.controls.reduce((sum, ctrl) => {
    const { quantity, price } = ctrl.value;
    return sum + (quantity * price || 0);
  }, 0);
}

createItem(): FormGroup {
  return this.fb.group({
    description: ['', Validators.required],
    quantity:    [1, [Validators.required, Validators.min(1)]],
    price:       [0, [Validators.required, Validators.min(0)]]
  });
}

```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone Angular component with a FormGroup containing three fields: firstName, lastName, and age. Add Validators.required to all fields and Validators.min(18) to age. Display a submit button that is disabled when the form is invalid, and log the form value to the console on valid submit.

Exercise 2 — Intermediate

Build a 'contact list' component using a FormArray of FormGroups. Each group has name (required) and phone (required, must be 10 digits). Allow users to dynamically add and remove contacts. Display the count of valid contacts above the list.

Exercise 3 — Advanced

Build a multi-step form wizard with three steps. Step 1: personal details (name, email). Step 2: address (street, city, postcode). Step 3: review and submit. Each step is its own FormGroup nested inside a parent FormGroup. Validate each step before allowing the user to proceed. On the review step, show all entered values in a read-only summary. Pre-populate the form via patchValue from a mock 'saved draft' object when the component initialises.

Section 10 — Key Takeaways

Core concepts in plain English:

- FormControl = one field. Tracks value, validity, dirty, touched.
- FormGroup = a named set of controls. Valid only if all children are valid.
- FormArray = a dynamic, index-based list of controls or groups.
- FormBuilder = a service shorthand that removes boilerplate.
- valueChanges = an Observable on any control/group/array. Subscribe with takeUntilDestroyed().
- patchValue = partial update (safe). setValue = full update (all fields required).

Method / Property	What it does
setValue(obj)	Sets entire form value — all fields required
patchValue(obj)	Sets partial form value — missing fields untouched
reset()	Clears value and marks pristine/untouched
getRawValue()	Gets value including disabled controls
markAllAsTouched()	Forces all error messages to show at once
disable() / enable()	Disables/enables the control
addControl() / removeControl()	Dynamically mutate a FormGroup
push() / removeAt()	Dynamically mutate a FormArray

Angular-specific rules:

- Always import ReactiveFormsModule in standalone component imports
- Subscribe to valueChanges in the constructor, not ngOnInit, when using takeUntilDestroyed()
- Use markAllAsTouched() in onSubmit() to reveal all validation errors
- Never trust [disabled] alone as a submit guard
- form.value omits disabled controls; use form.getRawValue() if you need them

One-Line Memory Aid: Reactive Forms are a TypeScript blueprint of your form — the template just connects to it; all logic, validation, and state live in the class.

Section 11 — Thought-Provoking Question + Answer

Question: If FormGroup.valid is computed from its children, and a FormArray is a child of a FormGroup, what happens to the parent form's validity when you dynamically push() an invalid control into the array?

Answer: Angular's form validity is a live tree. Every node computes its valid state bottom-up. When you push() a new FormControl(" ", Validators.required) into a FormArray, the new control

starts empty and invalid. It immediately marks its parent `FormArray` as invalid, which propagates to the root `FormGroup`. The root becomes invalid. If your submit button is bound to `[disabled]="form.invalid"`, it immediately disables — before the user has typed a single character into the new field. Practical rule: When adding items to a `FormArray`, either pre-populate with valid defaults or ensure your template validation feedback is gated on `.touched` — never on `.valid` alone.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts

- `Object.entries` / `Object.keys` — useful when iterating over form values to build API payloads
- Immutability patterns — patching form value from API without mutating original data

TypeScript Concepts

- Mapped Types / `Partial<T>` — understanding why `patchValue` accepts a `Partial` of your model
- Strict Typed Reactive Forms (`FormGroup<{...}>`) — TypeScript 4.7+ feature for full generics

Angular-Specific Concepts

- Topic 34 — Form Validation — the natural next step; validators are the most complex part of Reactive Forms
- Topic 35 — Template-Driven Forms — understand the contrast
- Topic 36 — `HttpClient` — `patchValue` from an API response is the most common real-world use
- `ControlValueAccessor` — how to build a reusable custom form control

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 13 (Interfaces), Topic 16 (Generics), Topic 20 (RxJS), Topic 21 (`takeUntilDestroyed`), Topic 27 (DI).

Unlocks: Topic 34 (Form Validation), Topic 35 (Template-Driven Forms), Topic 36 (`HttpClient`), Topic 39 (Route Guards — `CanDeactivate` checking `form.dirty`), Topic 47 (Testing).

Production readiness: If you can build a `FormArray` with dynamic `add/remove`, add validators, display conditional validation errors, and subscribe to `valueChanges` without a memory leak — you are production-ready for 90% of real Angular forms.

Section 14 — Common Interview Questions

Question 1 (Junior) — `setValue` vs `patchValue`

'What is the difference between `setValue` and `patchValue` in Reactive Forms, and when would you use each?' Strong answer: `setValue` requires all controls to be supplied — throws if any are missing. `patchValue` accepts a partial object. Use `setValue` for replacing the entire form; `patchValue` for pre-filling from a partial API response. Weak answer misses: That `setValue` throws on missing fields — not just ignores them.

Question 2 (Mid) — Dynamic form fields

'How do you handle dynamic form fields in Angular?' Strong answer: `FormArray` is the correct primitive. `push(control)` adds, `removeAt(index)` removes. `formArrayName` directive + `[formControlName]="$index"` for binding. Weak answer misses: The template binding

requirement.

Question 3 (Mid-Senior) — Cross-field validator

'How would you implement a cross-field validator — e.g. confirmPassword matches password?'

Strong answer: Applied at FormGroup level. Validator receives AbstractControl (the group) and calls .get('field'). Returns null or error object. Error lives on the group not the control. Weak answer misses: That the error is set on the group, not on the confirmPassword control.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Subscribing to valueChanges in ngOnInit without cleanup

```
// WRONG
ngOnInit() {
  this.searchForm.get('query')!.valueChanges.subscribe(val => {
    this.searchService.search(val).subscribe(results => {
      this.results = results;
    });
  });
}

// CORRECT
constructor() {
  this.searchForm.get('query')!.valueChanges.pipe(
    debounceTime(300),
    switchMap(val => this.searchService.search(val)),
    takeUntilDestroyed()
  ).subscribe(results => this.results = results);
}
```

Why it seems fine: Works during development. What breaks: Every navigation adds a new subscription. After 10 navigations, 10 active subscriptions fire simultaneously. Correct pattern: Use takeUntilDestroyed() and switchMap.

Anti-Pattern 2 — Accessing form.value without guarding or markAllAsTouched

```
// WRONG
onSubmit() { this.api.register(this.form.value).subscribe(); }

// CORRECT
onSubmit() {
  if (this.registrationForm.invalid) {
    this.registrationForm.markAllAsTouched();
    return;
  }
  this.api.register(this.registrationForm.value).subscribe();
}
```

Anti-Pattern 3 — Rebuilding the entire FormArray on every data change

```
// WRONG – wipes user's in-progress edits, fires valueChanges on every push
loadItems(items: Item[]) {
  const array = this.form.get('items') as FormArray;
  array.clear();
  items.forEach(item => array.push(this.fb.group({ name: item.name })));
}

// CORRECT – patch existing, add/remove only what changed
loadItems(items: Item[]) {
  const array = this.form.get('items') as FormArray;
  items.forEach((item, i) => {
    if (array.at(i)) {
      array.at(i).patchValue({ name: item.name }, { emitEvent: false });
    } else {
      array.push(this.fb.group({ name: item.name }, { emitEvent: false }));
    }
  });
  while (array.length > items.length) {
    array.removeAt(array.length - 1, { emitEvent: false });
  }
}
```

Section 16 — Mental Model Summary

Core concept in one sentence: Reactive Forms define your form as a TypeScript class tree — FormControl, FormGroup, FormArray — and the template simply binds to it; all state, validation, and reactivity live in the class.

Key rules:

- 1. FormControl = one field. FormGroup = named set. FormArray = indexed dynamic list.
- 2. Validity is a live tree: a single invalid child makes the entire parent invalid.
- 3. patchValue = partial update. setValue = full update (throws on missing fields).
- 4. Always subscribe to valueChanges in the constructor with takeUntilDestroyed().
- 5. Call markAllAsTouched() in onSubmit() to surface all hidden validation errors.
- 6. form.value excludes disabled controls — use form.getRawValue() to include them.

The single most important Angular rule: Never subscribe to valueChanges without takeUntilDestroyed() — every unmanaged form subscription is a memory leak waiting to happen in production.

Situation	Use
Pre-filling from a full API response	setValue
Pre-filling from a partial API response	patchValue
Updating one field programmatically	patchValue
Resetting the entire form	setValue or reset(defaultValues)

TOPIC 34 — Form Validation (Built-in, Custom, Async Validators)

Angular Core — Forms | Difficulty: Intermediate | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The built-in validators are straightforward, but custom synchronous validators require understanding `AbstractControl`, and async validators add RxJS observables into the mix, making this a multi-layered topic.

Angular Relevance: Critical — Every real-world form needs validation beyond basic required checks. Custom and async validators are what separates production-grade Angular forms from toy examples.

Section 1 — Explanation

The Bouncer Analogy

Imagine a nightclub with three types of bouncers at the door:

Bouncer Type 1 — The Rulebook Bouncer (Built-in Validators): Has a printed rulebook. Checks instantly: 'Are you over 18? Is your ID not expired?' Fast, synchronous, no waiting. Angular ships with: `required`, `email`, `minLength`, `maxLength`, `min`, `max`, `pattern`.

Bouncer Type 2 — The Custom Rulebook Bouncer (Custom Validators): You wrote the rules yourself. Still checks instantly — no waiting — but the rules are yours. This is a custom synchronous validator.

Bouncer Type 3 — The Radio Bouncer (Async Validators): Has to radio the VIP list database before letting someone in. Takes a moment. The door says 'checking...' (pending state) before the answer comes back. This is an async validator — it makes an HTTP call and returns an `Observable` or `Promise`.

All three bouncers return the same answer: `null` (you're in — valid) or an error object (you're not — here's why).

Every validator follows one contract:

```
ValidatorFn:      (control: AbstractControl) => ValidationErrors | null
AsyncValidatorFn: (control: AbstractControl) => Observable<ValidationErrors | null> | Promise<ValidationErrors | null>
```

Validator Type	Returns	When it runs	pending state?
Built-in (sync)	<code>ValidationErrors null</code>	On every value change	No
Custom sync	<code>ValidationErrors null</code>	On every value change	No
Custom async	<code>Observable<...> Promise<...></code>	After all sync validators pass	Yes — until resolved

Critical rule: Angular only runs async validators if all synchronous validators pass first. This prevents unnecessary HTTP calls when the field is obviously invalid.

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 33 — Reactive Forms: Validators are meaningless without FormControl/FormGroup to attach them to
- Topic 8 — Promises & async/await: Async validators can return Promises
- Topic 20 — RxJS Subscription Management: Async validators return Observables; Angular subscribes internally
- Topic 16 — TypeScript Generics: AbstractControl is a generic base class
- Topic 18 — Error Handling: Async validators must handle HTTP errors to avoid pending state getting stuck

Bridging note: Topic 33 introduced validators being passed into FormControl. This topic explains what those validators actually are — functions following a strict contract — and how to write your own. The async validator pattern is essentially Topic 8's async/await knowledge expressed as an RxJS Observable returned to Angular's form engine.

Extends most directly: Topic 33 — Reactive Forms.

Section 3 — Code Example

Plain TS — Validator contract in isolation

```
// A validator is just a function.
// It receives a control and returns null (valid) or an error object (invalid).

function noSpacesValidator(control: AbstractControl): ValidationErrors | null {
  const hasSpaces = (control.value as string)?.includes(' ');
  return hasSpaces ? { noSpaces: { actualValue: control.value } } : null;
}

const usernameControl = new FormControl('hello world', [noSpacesValidator]);
console.log(usernameControl.errors); // { noSpaces: { actualValue: 'hello world' } }
console.log(usernameControl.valid); // false
```

Custom Sync Validator — Parameterised factory

```
// Parameterised validator: returns a ValidatorFn
export function forbiddenNameValidator(forbiddenNames: string[]): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const isForbidden = forbiddenNames.includes(control.value?.toLowerCase());
    return isForbidden ? { forbiddenName: { value: control.value } } : null;
  };
}
```

Cross-Field Sync Validator (applied to FormGroup)

```
// Applied at group level – accesses sibling controls via control.get()
export function passwordMatchValidator(group: AbstractControl): ValidationErrors | null {
  const password = group.get('password')?.value;
  const confirmPassword = group.get('confirmPassword')?.value;
  return password === confirmPassword ? null : { passwordMismatch: true };
}
```

Async Validator — Username availability check

```

export function usernameAvailabilityValidator(userService: UserService): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    return timer(400).pipe(
      // debounce 400ms
      switchMap(() => userService.checkUsernameAvailable(control.value)),
      map(isAvailable => isAvailable ? null : { usernameTaken: true }),
      catchError(() => of(null)) // on HTTP error, treat as valid
    );
  };
}

// Registration in fb.group:
username: [
  '',
  [Validators.required, Validators.minLength(3), forbiddenNameValidator(['admin'])],
  [usernameAvailabilityValidator(this.userService)] // third argument = async validators
],

```

Template — showing all validator states

```

@if (username.pending) {
  <small>Checking availability...</small>
}
@if (username.invalid && username.touched && !username.pending) {
  <small>
    @if (username.errors?.['required']) { Username is required }
    @if (username.errors?.['minlength']) { Must be at least 3 characters }
    @if (username.errors?.['forbiddenName']) { That username is not allowed }
    @if (username.errors?.['usernameTaken']) { Username is already taken }
  </small>
}

<!-- Group-level error lives on the group, not the individual control -->
@if (registerForm.errors?.['passwordMismatch'] && confirmPassword.touched) {
  <small>Passwords do not match</small>
}

```

Section 4 — Before & After Refactor

Before — Inline validation logic in the component

```

// WRONG: Validation logic scattered inside onSubmit – not reactive
onSubmit() {
  const { username, email } = this.registerForm.value;
  if (!username || username.length < 3) {
    alert('Username must be at least 3 characters');
    return;
  }
  if (!email || !email.includes('@')) {
    alert('Invalid email');
    return;
  }
}

```

What breaks: No live feedback as user types. [disabled] binding can't work. Logic locked in onSubmit — impossible to unit test. Scaling to 10+ fields is unmanageable.

After — Validators declared on the form model

```

registerForm = this.fb.group({
  username: ['', [Validators.required, Validators.minLength(3)]],
  email:    ['', [Validators.required, Validators.email]]
});

onSubmit() {
  if (this.registerForm.invalid) {
    this.registerForm.markAllAsTouched();
    return;
  }
  // submit...
}

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Returning false or true instead of null or error object

What: Returning false/true from a validator. Why: Feels natural from native HTML validation.

How to avoid: Contract is strict: return null for valid, return { errorKey: true } for invalid.

Angular-specific consequence: Validator silently does nothing — field is always treated as valid.

Mistake 2 — Not debouncing async validators

What: Firing an HTTP request on every keystroke without `timer(400).pipe(switchMap(...))`. Why:

Happy path works in dev with fast APIs. How to avoid: Always wrap with

`timer(400).pipe(switchMap(...))`. Angular-specific consequence: Typing 'johndoe' fires 7 HTTP requests with race conditions.

Mistake 3 — Forgetting catchError in async validators

What: No `catchError` in the async validator Observable chain. Why: Error handling feels like an

edge case. How to avoid: Always pipe `catchError(() => of(null))`. Angular-specific consequence: If API returns 500, the Observable errors, Angular's subscription dies, control stays in PENDING state forever.

Section 6 — Do's and Don'ts

DO	DON'T
Return null for valid, { errorKey: true } for invalid — always	Return true, false, or undefined from a validator
Put reusable validators in a dedicated validators/ file	Inline one-off validator logic in the component class
Add <code>catchError(() => of(null))</code> to every async validator	Let async validators throw — they permanently lock the form in pending
Debounce async validators with <code>timer(400).pipe(switchMap(...))</code>	Fire an HTTP request on every single keystroke
Apply cross-field validators at the FormGroup level	Apply cross-field validators to an individual FormControl
Show pending state feedback while async validates	Leave users with no indication the form is checking something

Section 7 — Debugging Tips

Bug 1 — Form stays in pending state indefinitely

Symptom: Control status is permanently PENDING, spinner never disappears, submit stays

disabled. Cause: Async validator's Observable threw an error (uncaught HTTP error), Angular's

internal subscription unsubscribed, control never received a final value. Fix: Always add `catchError(() => of(null))` at the end of every async validator.

Bug 2 — Custom validator always returns invalid even with correct input

Symptom: Field shows errors even when value looks correct. Cause: Validator checks `control.value` before it has a value (null or "") and null check is missing. Fix: Guard for empty/null — `if (!control.value) return null`. Let `Validators.required` handle the empty case.

Bug 3 — Cross-field validation error doesn't show in template

Symptom: Group-level validator fires but error message never appears. Cause: Developer checks `form.get('confirmPassword').errors?.['passwordMismatch']` — but the error is on the GROUP, not the individual control. Fix: Check `form.errors?.['passwordMismatch']` — group-level errors live on the group.

Section 8 — Real-World Applications

Scenario 1 — Password Strength Validator with Detailed Feedback

```
export function passwordStrengthValidator(control: AbstractControl): ValidationErrors | null {
  const value: string = control.value || '';
  if (!value) return null;

  const errors: ValidationErrors = {};
  if (value.length < 8) errors['minLength'] = true;
  if (!/[A-Z]/.test(value)) errors['requiresUpper'] = true;
  if (!/[0-9]/.test(value)) errors['requiresNumber'] = true;
  if (!/[!@#$$%^&*]/.test(value)) errors['requiresSpecial'] = true;

  return Object.keys(errors).length > 0 ? errors : null;
}
```

Scenario 2 — Async Email Uniqueness Validator

```
export function emailUniqueValidator(
  authService: AuthService,
  currentEmail: string
): AsyncValidatorFn {
  return (control: AbstractControl): Observable<ValidationErrors | null> => {
    if (control.value === currentEmail) return of(null); // skip check for own email
    return timer(500).pipe(
      switchMap(() => authService.isEmailTaken(control.value)),
      map(taken => taken ? { emailTaken: true } : null),
      catchError(() => of(null))
    );
  };
}
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone Angular component with a single `FormControl` for a UK postcode. Write a custom synchronous validator that checks the value against a basic UK postcode pattern: must start with 1-2 letters followed by 1-2 numbers. Display 'Invalid UK postcode format' below the input when validation fails, but only after the user has touched the field.

Exercise 2 — Intermediate

Build a 'create account' form with username, email, password, and confirmPassword fields. Write: (a) a custom sync validator ensuring username contains no uppercase letters, (b) a cross-field group validator ensuring password and confirmPassword match, (c) proper validation error messages for all fields shown only after touched. Submit button disabled when invalid or pending.

Exercise 3 — Advanced

Build an email settings component where a user can update their email. The form has a single email field. Write an async validator that simulates an API check (use `of(true/false)` with `timer(600)` delay). Handle three states: pending (show 'Checking...'), emailTaken error, and valid (green checkmark). The validator must skip the API call if the email matches the current email, and must handle errors gracefully without locking the form.

Section 10 — Key Takeaways

Core concepts:

- Every validator is a function: takes a control, returns null (valid) or { errorKey: value } (invalid)
- Built-in validators: required, email, minLength, maxLength, min, max, pattern
- Custom sync validators: plain functions, run on every value change
- Custom async validators: return Observable or Promise, run only after sync validators pass, have a pending state
- Cross-field validators: applied at FormGroup level — errors land on the group, not the control
- `catchError(() => of(null))` is mandatory in every async validator

	Sync Validator	Async Validator
Return type	ValidationErrors null	Observable
When it runs	Every value change	After all sync validators pass
pending state	No	Yes — while Observable is resolving
Placement in fb.group	2nd argument array	3rd argument array
Typical use	Format, length, pattern	API calls (username taken, email exists)

Angular-specific rules:

- Async validators only run when all sync validators pass — add `Validators.required` before async validators
- Cross-field errors live on the group — template must check `form.errors?.['key']`
- Always `catchError(() => of(null))` in async validators — no exceptions
- Always debounce async validators with `timer(N).pipe(switchMap(...))`
- Show `form.pending` in the template and disable submit while async validation is in progress

One-Line Memory Aid: A validator is just a function that returns null for pass or { errorKey: true } for fail — built-in, custom sync, and async all follow the exact same contract.

Section 11 — Thought-Provoking Question + Answer

Question: Angular only runs async validators after all synchronous validators pass. But what if you want an async validator to run even when the field is empty — to show 'this field cannot be empty' from a server-side rule?

Answer: Don't fight the framework's optimisation — layer validators correctly. If the server enforces required, add `Validators.required` as a sync validator too. It's not duplication — the sync validator provides immediate, offline feedback while the async validator provides authoritative, server-side validation. For dynamic validation configs, load the validation config from the server before building the form, then construct validators programmatically. Practical rule: Always add a sync `Validators.required` guard before any async validator that should not run on empty input.

Section 12 — Related Topics to Learn Next

ES6/JS Concepts

- Regular expressions — needed for `Validators.pattern` and custom pattern validators
- Closures — parameterised validators (factory functions) are closures capturing outer arguments

TypeScript Concepts

- Function types — understanding `ValidatorFn` and `AsyncValidatorFn` as typed function signatures
- `Partial<T>` and mapped types — how Angular types partial form values internally

Angular-Specific Concepts

- Topic 35 — Template-Driven Forms: validators work differently (directive-based)
- Topic 36 — `HttpClient`: required for real async validators that check API endpoints
- `updateOn` configuration ('change' | 'blur' | 'submit') — controls when validators run

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 8 (Promises), Topic 20 (RxJS), Topic 33 (Reactive Forms).

Unlocks: Topic 35 (Template-Driven Forms), Topic 36 (`HttpClient`), Topic 39 (Route Guards — `CanDeactivate`), Topic 47 (Testing).

Production readiness: If you can write a parameterised custom sync validator, a debounced async validator with error handling, and a cross-field group validator — and correctly display their errors in a template — you are fully production-ready for Angular form validation.

Section 14 — Common Interview Questions

Question 1 (Junior) — Sync vs Async validators

'What is the difference between a synchronous and an asynchronous validator?' Strong answer: Sync validators run immediately on every change; async validators return `Observable`/`Promise` and run only after sync validators pass. The pending status exists only during async validation. Use sync for format/length/pattern; use async for server-side uniqueness checks. Always debounce async validators. Weak answer misses: That Angular intentionally skips async validators when sync validators fail.

Question 2 (Mid) — Cross-field validator

'How do you implement a cross-field validator?' Strong answer: Applied at `FormGroup` level. Receives the group as `AbstractControl`, calls `.get('fieldName')` to access children. Returns null or

error object. Error lands on the group's `.errors`. Template checks `form.errors?.['errorKey']`. Weak answer misses: Where the error actually lands — the group, not the child.

Question 3 (Senior) — Submit button stays disabled after network error

'A user reports that after a network error, the form's submit button stays permanently disabled. What would you investigate?' Strong answer: Root cause is async validator's Observable threw an error — Angular's internal subscription died, control stuck in PENDING. `catchError(() => of(null))` is the fix. Treating API error as valid (`of(null)`) is correct safe default. Weak answer misses: The mechanism — understanding that Observable completing vs erroring drives status transition out of PENDING.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Async validator without debounce or switchMap

```
// WRONG – fires on every keystroke
export function checkEmail(service: AuthService): AsyncValidatorFn {
  return (control) => service.isEmailTaken(control.value).pipe(
    map(taken => taken ? { emailTaken: true } : null)
  );
}

// CORRECT
export function checkEmail(service: AuthService): AsyncValidatorFn {
  return (control) => timer(400).pipe(
    switchMap(() => service.isEmailTaken(control.value)),
    map(taken => taken ? { emailTaken: true } : null),
    catchError(() => of(null))
  );
}
```

Anti-Pattern 2 — Manually running validators in `onSubmit`

```
// WRONG
onSubmit() {
  const { username } = this.form.value;
  if (!username) { this.errors.push('Username required'); return; }
  // logic duplicated from form validators
}

// CORRECT
onSubmit() {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  this.api.register(this.form.value).subscribe();
}
```

Anti-Pattern 3 — Validator closing over mutable state

```
// WRONG – mutable closure, validator doesn't re-evaluate when minLength changes
export class SearchComponent {
  private minLength = 3;
  myValidator = (control: AbstractControl) => {
    return control.value?.length >= this.minLength ? null : { tooShort: true };
  };
}

// CORRECT – replace the validator when config changes
this.form.get('search')?.setValidators([
  Validators.minLength(this.configService.getMinLength())
]);
this.form.get('search')?.updateValueAndValidity();
```

Section 16 — Mental Model Summary

Core concept in one sentence: A validator is a function that receives a control and returns null (valid) or an error object (invalid) — built-in, custom sync, and async all follow this exact contract; only the return type and timing differ.

Key rules:

- 1. Return null = valid. Return { errorKey: true } = invalid. Nothing else is valid.
- 2. Sync validators run on every change. Async validators run only after all sync validators pass.
- 3. Cross-field validators go on the FormGroup — errors land on the group, not the child controls.
- 4. Every async validator needs `catchError(() => of(null))` — no exceptions.
- 5. Debounce async validators with `timer(400).pipe(switchMap(...))`.
- 6. Call `markAllAsTouched()` in `onSubmit()` to surface all hidden errors at once.

The single most important Angular rule: Every async validator must end with `catchError(() => of(null))` — a thrown error permanently locks the control in PENDING state and disables the form.

Validator type	Where to attach	Where error appears
Single-field sync	2nd arg of fb.group field	control.errors
Single-field async	3rd arg of fb.group field	control.errors
Cross-field sync	validators option of fb.group	formGroup.errors
Cross-field async	asyncValidators option of fb.group	formGroup.errors

TOPIC 35 — Template-Driven Forms

Angular Core — Forms | Difficulty: Beginner-Intermediate | Angular Relevance: High

Section 0 — Difficulty & Priority Rating

Difficulty: Beginner-Intermediate — The template syntax is approachable, but understanding why it works — and where it breaks down compared to Reactive Forms — requires a solid mental model of Angular's directive system binding to an implicit form model behind the scenes.

Angular Relevance: High — Template-Driven Forms are the right tool for simple, low-logic forms. You'll encounter them in legacy codebases, third-party libraries, and anywhere rapid prototyping matters.

Section 1 — Explanation

The Verbal Instructions Analogy

Recall the Blueprint Analogy from Topic 33. Template-Driven Forms are the other approach — the Verbal Instructions approach. Instead of drawing a blueprint in TypeScript before construction begins, you walk onto the site and tell Angular what to build by annotating the HTML directly: 'This input is called username. Make it required. Minimum 3 characters.' Angular listens, builds the form model for you behind the scenes, and keeps it in sync with the template.

Simple shed? Verbal instructions work great. Complex skyscraper with dynamic floors? You'll wish you had a blueprint. That's when Reactive Forms win.

Template-Driven Forms are Angular saying: 'You describe the form in HTML — I'll create the model.' Reactive Forms are you saying: 'I'll create the model in TypeScript — you just connect the template to it.'

Technical explanation: When you add FormsModule and use `ngModel` on an input, Angular automatically: (1) creates a `FormControl` behind the scenes, (2) creates a `FormGroup` for the enclosing form element via the `NgForm` directive, (3) registers each `ngModel` control into that implicit group using the `name` attribute, (4) keeps the DOM value and implicit model in sync via two-way binding.

	Template-Driven	Reactive
Form model created by	Angular (implicit, in template)	You (explicit, in TypeScript)
Primary module	FormsModule	ReactiveFormsModule
Two-way binding	<code>[(ngModel)]</code> — built in	Manual (<code>formControlName</code> + event)
Validation	HTML attributes + directives	Validator functions in TypeScript
Access in component	Via <code>@ViewChild</code> or template ref	Direct property access
Testability	Harder (requires DOM)	Easy (pure TypeScript)
Best for	Simple, low-logic forms	Complex, dynamic, testable forms

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 33 — Reactive Forms: Template-Driven Forms create the same FormControl/FormGroup tree, just implicitly
- Topic 26 — @Input() & @Output(): [(ngModel)] is syntactic sugar for [ngModel] + (ngModelChange) — the banana-in-a-box pattern
- Topic 28 — Angular Template Syntax: ngModel, ngForm, ngSubmit are all template directives
- Topic 25 — Component Lifecycle Hooks: ngAfterViewInit is needed if you access NgForm via @ViewChild
- Topic 32 — ViewChild: The primary way to access the implicit form model from the component class

Extends most directly: Topic 33 — Reactive Forms, because Template-Driven Forms use the same underlying model (FormControl, FormGroup) surfaced through a different API.

Section 3 — Code Example

Plain concept — what [(ngModel)] actually desugars to

```
<!-- These two are identical: -->

<!-- Shorthand (banana-in-a-box) -->
<input [(ngModel)]="username" />

<!-- Longhand (what Angular actually executes) -->
<input [ngModel]="username" (ngModelChange)="username = $event" />
```

Real Angular Component — Complete Template-Driven Form

```

@Component({
  selector: 'app-contact-form',
  standalone: true,
  imports: [FormsModule, CommonModule],
  template: `
    <!-- #contactForm="ngForm" creates a template reference to NgForm directive -->
    <form #contactForm="ngForm" (ngSubmit)="onSubmit(contactForm)">

      <!-- name attribute is REQUIRED – registers control in NgForm -->
      <input name="name" [(ngModel)]="formData.name"
        required minlength="2" #nameField="ngModel" />

      <!-- #nameField="ngModel" gives access to .valid, .invalid, .touched -->
      @if (nameField.invalid && nameField.touched) {
        <small>
          @if (nameField.errors?.['required']) { Name is required }
          @if (nameField.errors?.['minlength']) { Minimum 2 characters }
        </small>
      }

      <input name="email" type="email" [(ngModel)]="formData.email"
        required email #emailField="ngModel" />

      <button type="submit" [disabled]="contactForm.invalid">Send</button>
    </form>
  `
})
export class ContactFormComponent {
  formData = { name: '', email: '', message: '', consent: false };

  onSubmit(form: NgForm): void {
    if (form.invalid) return;
    console.log('Submitted:', this.formData);
    form.reset(); // clears values AND resets dirty/touched/pristine
  }
}

```

Accessing NgForm from the component class via @ViewChild

```

export class EditProfileComponent implements AfterViewInit {
  @ViewChild('profileForm') profileForm!: NgForm;
  bio = '';

  ngAfterViewInit() {
    // Use setTimeout to avoid ExpressionChangedAfterItHasBeenChecked
    setTimeout(() => {
      this.profileForm.setValue({ bio: 'Loaded from server' });
    });
  }

  onSubmit() {
    if (this.profileForm.invalid) return;
    console.log(this.profileForm.value);
  }
}

```

Section 4 — Before & After Refactor

Before — Missing name attribute, broken registration

```

<!-- WRONG: no name attribute – Angular can't register this control in NgForm -->
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <input [(ngModel)]="email" required />
  <button type="submit">Submit</button>
</form>
<!-- Angular throws: If ngModel is used within a form tag, either the name
      attribute must be set or the form control must be defined as 'standalone' -->

```

After — Correct name attribute added

```

<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <!-- name="email" registers this control under the key 'email' in NgForm -->
  <input name="email" [(ngModel)]="email" required #emailRef="ngModel" />
  @if (emailRef.invalid && emailRef.touched) {
    <small>Email is required</small>
  }
  <button type="submit" [disabled]="myForm.invalid">Submit</button>
</form>

```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Forgetting the name attribute on ngModel inputs

What: Using `[(ngModel)]` without `name='fieldName'` inside a form element. Why: `[(ngModel)]` works fine outside a form (standalone two-way binding), so developers assume it works the same inside one. How to avoid: Every `[(ngModel)]` inside a form must have a name attribute. Angular-specific consequence: Angular throws a runtime error, or `NgForm.value` object is empty.

Mistake 2 — Trying to access NgForm before ngAfterViewInit

What: `@ViewChild('myForm') myForm!: NgForm` accessed in `ngOnInit`. Why: `ngOnInit` is habitually 'where setup happens.' How to avoid: Always use `@ViewChild`-based `NgForm` access in `ngAfterViewInit` and wrap `setValue/patchValue` calls in `setTimeout()`. Angular-specific consequence: `TypeError: Cannot read properties of undefined`.

Mistake 3 — Confusing ngModel with FormControlName

What: Mixing `FormsModule` directives (`ngModel`) with `ReactiveFormsModule` directives (`FormControlName`) in the same form. Why: Both achieve input binding. How to avoid: Pick one system per form. Angular-specific consequence: Angular throws 'ngModel cannot be used to register form controls with a parent formGroup directive'.

Section 6 — Do's and Don'ts

DO	DON'T
Import <code>FormsModule</code> in standalone component imports	Import <code>ReactiveFormsModule</code> for Template-Driven Forms — wrong module
Always add <code>name='fieldName'</code> to every <code>ngModel</code> input inside a form	Use <code>ngModel</code> inside a form without a name attribute
Use template reference variables (<code>#field='ngModel'</code>) for per-field errors	Try to access field state from the class without <code>@ViewChild</code>
Pass the <code>NgForm</code> reference into <code>onSubmit(form: NgForm)</code> via the template	Ignore the <code>NgForm</code> reference and manually validate in submit handler

DO	DON'T
Use <code>form.reset()</code> after successful submit to clear state and mark pristine	Manually clear <code>formData</code> properties — leaves dirty/touched state stale
Use Reactive Forms for complex, dynamic, testable forms	Use Template-Driven Forms for forms with dynamic fields or async checks

Section 7 — Debugging Tips

Bug 1 — `ngModel` not updating the component property

Symptom: Input renders correctly but `this.formData.name` never updates. Cause: (a) `FormsModule` not imported — `ngModel` silently degrades; (b) developer used `[ngModel]` (one-way) instead of `[(ngModel)]` (two-way). Fix: Ensure `FormsModule` is imported and use `[(ngModel)]` syntax.

Bug 2 — `ExpressionChangedAfterItHasBeenChecked` when pre-populating

Symptom: Console error `ExpressionChangedAfterItHasBeenChecked` when setting form value in `ngAfterViewInit`. Cause: Angular runs change detection after `ngAfterViewInit`; synchronous value changes during that hook violate unidirectional data-flow in dev mode. Fix: Wrap `setValue` in `setTimeout()` to push update to the next change detection cycle.

Bug 3 — Validation errors not showing even when field is invalid

Symptom: Field is invalid but `@if (field.invalid && field.touched)` never renders. Cause: Template reference variable written as `#nameField` without `=ngModel` — this creates a ref to the DOM element, not the `NgModel` directive. Fix: Always write `#nameField=ngModel` (with `=ngModel`).

Section 8 — Real-World Applications

Scenario 1 — Simple Newsletter Signup Form

```
<form #signupForm="ngForm" (ngSubmit)="onSignup(signupForm)">
  <input name="email" type="email" [(ngModel)]="email"
    required email placeholder="your@email.com" #emailRef="ngModel" />

  @if (emailRef.invalid && (emailRef.touched || signupForm.submitted)) {
    <small>
      @if (emailRef.errors?.['required']) { Email is required }
      @if (emailRef.errors?.['email'])    { Enter a valid email }
    </small>
  }

  <button type="submit" [disabled]="signupForm.invalid || submitted">
    {{ submitted ? 'Subscribed!' : 'Subscribe' }}
  </button>
</form>
```

Scenario 2 — Edit Profile Form Pre-Populated from a Service

```

export class EditProfileComponent implements AfterViewInit {
  @ViewChild('profileForm') profileForm!: NgForm;
  profile = { displayName: '', bio: '' };
  private userService = inject(UserService);

  ngAfterViewInit() {
    this.userService.getProfile().subscribe(data => {
      setTimeout(() => {
        this.profileForm.setValue({
          displayName: data.displayName,
          bio: data.bio ?? ''
        });
      });
    });
  }

  onSave(form: NgForm) {
    if (form.invalid) return;
    this.userService.updateProfile(this.profile).subscribe(() => {
      form.markAsPristine(); // re-disable Save – no unsaved changes
    });
  }
}

```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a standalone Angular component with a Template-Driven login form containing email and password fields. Both required; email must be valid format; password at least 6 characters. Show inline error messages only after touched. Submit button disabled when invalid. Log form value on valid submit and reset the form.

Exercise 2 — Intermediate

Build a 'feedback' form with four fields: name (required), rating (select 1-5, required), category (radio buttons: Bug/Feature/Other, required), and comment (textarea, required, min 20 chars). Display a live character count showing how many characters remain. Disable submit when invalid and show all errors if user submits without touching fields (hint: use `ngForm.submitted`).

Exercise 3 — Advanced

Build an 'edit account' form that pre-populates from a mock service using `@ViewChild` and `ngAfterViewInit`. Fields: username, email, timezone (select). After a successful 'save' (simulate with `setTimeout`), mark the form as pristine so the Save button re-disables. Implement an unsaved-changes guard: if `profileForm.dirty` is true and user clicks 'Cancel', show a `confirm()` dialog. Ensure correct lifecycle hook usage and avoid `ExpressionChangedAfterItHasBeenChecked` errors.

Section 10 — Key Takeaways

Core concepts:

- Template-Driven Forms let Angular build the form model from HTML — you annotate, Angular constructs
- `FormsModule` must be imported — it provides `NgForm`, `NgModel`, and built-in validation directives
- Every `ngModel` inside a form requires a name attribute — it's the registration key

- `#formRef='ngForm'` and `#fieldRef='ngModel'` give access to form/control state in the template
- `[(ngModel)]` is two-way binding: `[ngModel]` sets value, `(ngModelChange)` writes back
- Validation applied via HTML attributes: `required`, `minlength`, `maxlength`, `email`, `pattern`

Task	Template-Driven syntax
Two-way bind an input	<code>[(ngModel)]='property'</code>
Register control in NgForm	<code>name='fieldName' (required)</code>
Access form state in template	<code>#formRef='ngForm'</code>
Access control state in template	<code>#fieldRef='ngModel'</code>
Check if field is invalid + touched	<code>fieldRef.invalid && fieldRef.touched</code>
Access specific error	<code>fieldRef.errors?.['required']</code>
Submit handler	<code>(ngSubmit)='onSubmit(formRef)'</code>
Reset form	<code>formRef.reset()</code>
Access NgForm in class	<code>@ViewChild('formRef') form!: NgForm</code>

One-Line Memory Aid: Template-Driven Forms are Angular building the model for you from your HTML — you annotate, Angular constructs; the trade-off is less control, less testability, but much less code for simple cases.

Section 11 — Thought-Provoking Question + Answer

Question: Both Template-Driven and Reactive Forms ultimately use the same `FormControl` and `FormGroup` classes under the hood. If the underlying model is identical, why does testing Template-Driven Forms require the DOM while Reactive Forms can be tested in pure TypeScript?

Answer: The difference isn't in the model — it's in who owns it and when it exists. In Reactive Forms, you create the `FormGroup` as a class property. It exists the moment the component class is instantiated — before any template, before any DOM. In Template-Driven Forms, Angular creates the `FormControl` objects as it parses and renders the template. The `NgModel` directive, which creates the control, only runs during Angular's view initialisation pass. The form model has a DOM dependency by construction. To unit test a Template-Driven Form, you need `TestBed.createComponent()`, `fixture.detectChanges()`, and DOM interactions to trigger `ngModel` registration. Reactive Forms need none of this. Practical rule: If a form will ever need automated unit tests, use Reactive Forms.

Section 12 — Related Topics to Learn Next

- Topic 36 — `HttpClient`: the natural next step after forms is submitting form data to an API
- Topic 39 — Route Guards (`CanDeactivate`): using `form.dirty` to prevent unsaved-changes navigation
- `ngModelOptions`: `updateOn 'blur' | 'submit'` controls when `ngModel` syncs
- `ControlValueAccessor`: building a custom input component that works with both `ngModel` and `formControlName`

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 25 (Lifecycle Hooks), Topic 26 (@Input/@Output), Topic 28 (Template Syntax), Topic 32 (ViewChild), Topic 33 (Reactive Forms).

Unlocks: Topic 36 (HttpClient), Topic 39 (Route Guards), Topic 47 (Testing — understanding why TDF tests need the DOM).

Production readiness: Template-Driven Forms are production-ready for simple, static forms. If you need dynamic fields, cross-field validators, async validators, or unit tests, switch to Reactive Forms. If you're unsure which to use, use Reactive — it scales; Template-Driven doesn't.

Section 14 — Common Interview Questions

Question 1 (Junior-Mid) — Template-Driven vs Reactive

'What is the difference between Template-Driven and Reactive Forms? When would you choose one over the other?' Strong answer: Template-Driven — implicit model, Angular creates FormControl/FormGroup from template. Reactive — explicit model, developer creates tree in TypeScript. Reactive Forms easier to unit test, support dynamic fields, scale better. Template-Driven faster for simple static forms. Both use same AbstractControl model. Weak answer misses: That both systems use the same underlying FormControl/FormGroup classes.

Question 2 (Junior) — Field validation state in template

'How do you access a specific field's validation state in the template?' Strong answer: Template reference variable with `=ngModel`: `#emailField=ngModel`. Without `=ngModel`, the ref points to the DOM element. Access errors via `emailField.errors?.['required']`, `emailField.touched`, `emailField.invalid`. The name attribute must be present. Weak answer misses: The distinction between `#ref` (DOM element) and `#ref=ngModel` (NgModel directive instance).

Question 3 (Mid) — Pre-populating a Template-Driven Form

'How would you pre-populate a Template-Driven Form with data loaded from an API?' Strong answer: `@ViewChild` to get class-level access. `NgForm` only available after `ngAfterViewInit`. Wrap `setValue/patchValue` in `setTimeout()`. Subscribe to API call in `ngAfterViewInit`, apply inside `setTimeout`. After saving, call `form.markAsPristine()`. Weak answer misses: The `setTimeout` requirement and why it's needed.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Using Template-Driven Forms for dynamic field lists

```
<!-- WRONG: dynamic fields in Template-Driven Form -->
@for (field of dynamicFields; track field.id) {
  <input [name]="field.id" [(ngModel)]="field.value" [required]="field.required" />
}

<!-- CORRECT: use Reactive Forms with FormArray for dynamic fields -->
<div formArrayName="fields">
  @for (ctrl of fields.controls; track $index) {
    <input [formControlName]="$index" />
  }
</div>
```

Why it seems fine: Angular registers each `ngModel` dynamically as the loop renders. What breaks: Removing a field doesn't cleanly deregister it from `NgForm`. Validators bound via `[required]` don't re-evaluate when required changes. `NgForm.value` may contain stale keys.

Anti-Pattern 2 — Relying on [disabled] as the only submit guard

```
// WRONG — no guard in onSubmit
onSubmit() {
  this.api.contact(this.formData).subscribe(); // always runs
}

// CORRECT
onSubmit(form: NgForm) {
  if (form.invalid) {
    form.control.markAllAsTouched();
    return;
  }
  this.api.contact(this.formData).subscribe();
}
```

Anti-Pattern 3 — Mixing [(ngModel)] with Reactive Form directives

```
<!-- WRONG: mixing both form systems -->
<form [formGroup]="myForm">
  <input formControlName="email" /> <!-- Reactive -->
  <input name="phone" [(ngModel)]="phone" /> <!-- Template-Driven — conflicts! -->
</form>
<!-- Angular throws: ngModel cannot be used with formControlName -->

<!-- CORRECT: pick one system per form -->
```

Section 16 — Mental Model Summary

Core concept in one sentence: Template-Driven Forms let Angular build the form model implicitly from your HTML using ngModel and NgForm directives — you annotate the template, Angular constructs the FormControl/FormGroup tree behind the scenes.

Key rules:

1. Import FormsModule (not ReactiveFormsModule) for Template-Driven Forms.
2. Every [(ngModel)] inside a form MUST have a name attribute — it's the registration key.
3. Use #ref='ngModel' (not just #ref) to access control state in the template.
4. NgForm is only available after ngAfterViewInit — never access it in ngOnInit.
5. Wrap setValue/patchValue calls in setTimeout() inside ngAfterViewInit.
6. Always add a validity guard in onSubmit — never trust [disabled] alone.

The single most important Angular rule: Template-Driven Forms do not scale — use them only for simple, static, low-logic forms; switch to Reactive Forms the moment you need dynamic fields, async validation, cross-field rules, or unit tests.

Signal	Use Template-Driven	Use Reactive
Form complexity	Simple, static fields	Dynamic, conditional fields
Validation needs	Built-in HTML attributes	Custom, cross-field, async
Testing requirement	Manual / e2e only	Unit testable
Code volume preference	Minimal TypeScript	More TS, cleaner at scale

TOPIC 36 — HttpClient & API Integration

Angular Core — HTTP & Routing | Difficulty: Intermediate | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The basic get/post calls are straightforward, but understanding typed responses, error handling, request configuration, and the interplay with RxJS operators is what separates production-quality HTTP code from tutorial-level code.

Angular Relevance: Critical — Almost every Angular app communicates with a backend. HttpClient is the official Angular mechanism for all HTTP communication. It is RxJS-native, interceptor-aware, and testable by design.

Section 1 — Explanation

The Postal Service Analogy

Imagine your Angular component is an office worker who needs to send and receive parcels from a warehouse (your API server). HttpClient is the postal service. You hand it a parcel (a request — URL, method, body, headers) and it delivers the response back to you. You don't manage the trucks, the routing, or the network.

But here's the key: the postal service doesn't hand you the parcel directly. It hands you a tracking slip — an Observable. The parcel (the HTTP response) hasn't arrived yet. You subscribe to the tracking slip, and when the parcel arrives, your callback runs.

Three important rules: (1) No subscription = no request — the Observable is cold. (2) Each subscription = a new request. (3) Unsubscribing = cancelling the parcel — if you unsubscribe before the response arrives, Angular cancels the in-flight request.

Method	HTTP verb	When to use
<code>http.get(url, options?)</code>	GET	Fetch data
<code>http.post(url, body, options?)</code>	POST	Create a resource
<code>http.put(url, body, options?)</code>	PUT	Replace a resource fully
<code>http.patch(url, body, options?)</code>	PATCH	Partially update a resource
<code>http.delete(url, options?)</code>	DELETE	Remove a resource
<code>http.request(method, url, options?)</code>	Any	Full control

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 8 — Promises & async/await: HttpClient returns Observables, not Promises; `firstValueFrom()` bridges them when needed
- Topic 20 — RxJS Subscription Management: every `http.get()` is an Observable that needs subscription lifecycle management
- Topic 21 — `takeUntilDestroyed` & `DestroyRef`: HTTP subscriptions in components must be cleaned up
- Topic 27 — Services & DI: HttpClient is always injected; API calls always live in services, not components

- Topic 33 — Reactive Forms: `patchValue` from an API response is the most common real-world use
- Topic 18 — Error Handling: HTTP errors are the most common real-world errors in Angular apps

Extends most directly: Topic 20 — RxJS Subscription Management, because `HttpClient` is RxJS-native and every real HTTP pattern is an RxJS problem.

Section 3 — Code Example

Plain concept — cold Observable, no request without subscription

```
// Creating the Observable does NOT fire the request
const request$ = http.get('https://api.example.com/users');

// The request fires HERE – when something subscribes
request$.subscribe(data => console.log(data));

// Subscribing again fires a SECOND independent request
request$.subscribe(data => console.log('second call:', data));
```

Real Angular Service — Typed, with error handling

```
export interface User { id: number; name: string; email: string; role: 'admin' | 'user'; }

@Injectable({ providedIn: 'root' })
export class UserService {
  private http = inject(HttpClient);
  private baseUrl = 'https://api.example.com';

  // GET with query params
  getUsers(page = 1, pageSize = 20): Observable<PaginatedResponse<User>> {
    const params = new HttpParams().set('page', page).set('pageSize', pageSize);
    return this.http.get<PaginatedResponse<User>>(`${this.baseUrl}/users`, { params })
      .pipe(catchError(this.handleError));
  }

  // POST: create a resource
  createUser(payload: Omit<User, 'id'>): Observable<User> {
    return this.http.post<User>(`${this.baseUrl}/users`, payload)
      .pipe(catchError(this.handleError));
  }

  // DELETE
  deleteUser(id: number): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/users/${id}`)
      .pipe(catchError(this.handleError));
  }

  private handleError(error: HttpResponse): Observable<never> {
    let msg = 'An unexpected error occurred';
    if (error.status === 0) msg = 'Network error – check your connection';
    if (error.status === 401) msg = 'Unauthorised – please log in again';
    if (error.status === 404) msg = 'Resource not found';
    if (error.status >= 500) msg = 'Server error – please try again later';
    console.error('HTTP Error:', error);
    return throwError(() => new Error(msg));
  }
}
```

Component — loading and error states with DestroyRef

```
export class UsersListComponent implements OnInit {
  private userService = inject(UserService);
  private destroyRef = inject(DestroyRef);

  users: User[] = [];
  loading: boolean = false;
  error: string | null = null;

  ngOnInit(): void { this.loadUsers(); }

  loadUsers(): void {
    this.loading = true;
    this.error = null;

    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef)) // works in any method with explicit DestroyRef
      .subscribe({
        next: response => { this.users = response.data; this.loading = false; },
        error: err => { this.error = err.message; this.loading = false; }
      });
  }
}
```

Section 4 — Before & After Refactor

Before — HTTP calls in component, untyped, no error handling

```
// WRONG
export class BadComponent implements OnInit {
  private http = inject(HttpClient); // HttpClient in component – not reusable
  users: any[] = []; // typed as any

  ngOnInit() {
    this.http.get('https://api.example.com/users').subscribe((data: any) => {
      this.users = data; // no error handling, no loading state, no cleanup
    });
  }
}
```

After — Typed service, error handling, loading state, cleanup


```
// CORRECT
export class GoodComponent implements OnInit {
  private userService = inject(UserService);
  private destroyRef = inject(DestroyRef);

  users: User[] = [];
  loading = false;
  error: string | null = null;

  ngOnInit() { this.loadUsers(); }

  loadUsers() {
    this.loading = true;
    this.userService.getUsers()
      .pipe(takeUntilDestroyed(this.destroyRef))
      .subscribe({
        next: res => { this.users = res.data; this.loading = false; },
        error: err => { this.error = err.message; this.loading = false; }
      });
  }
}
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Not subscribing and wondering why nothing happens

What: `this.userService.createUser(this.formData)` — forgot `.subscribe()`. Why: Coming from Promises, where calling `fetch(...)` immediately fires. With Observables, the call is lazy. How to avoid: Always subscribe, or use `async pipe` / `toSignal()` to subscribe automatically. Angular-specific consequence: API call silently never happens — no error, no feedback.

Mistake 2 — Putting `HttpClient` calls directly in components

What: Injecting `HttpClient` into a component and calling `this.http.get(...)` directly. Why: Faster — fewer files. How to avoid: Always create a dedicated service. Angular-specific consequence: HTTP logic can't be reused, can't be mocked with `HttpClientTestingModule`, interceptors don't work cleanly.

Mistake 3 — Ignoring the error callback in `.subscribe()`

What: Only providing a next callback, no error handler. Why: Happy path works in dev. How to avoid: Always provide error callback in `.subscribe({ next, error })` or `.pipe(catchError(...))`. Angular-specific consequence: API failures leave component in permanent loading or empty state.

Section 6 — Do's and Don'ts

DO	DON'T
Inject <code>HttpClient</code> in services — never in components	Call <code>http.get()</code> inside a component class
Type every response: <code>http.get(url)</code>	Use <code>any</code> as the response type
Always handle errors — <code>catchError</code> in service, error callback in component	Provide only a next callback and ignore errors
Use <code>HttpParams</code> to build query strings	Manually concatenate query strings: <code>url + '?page=' + page</code>
Use <code>takeUntilDestroyed(destroyRef)</code> for component subscriptions	Let HTTP subscriptions run indefinitely without cleanup

DO	DON'T
Set headers via interceptors (Topic 37), not per-call	Add auth headers manually to every http.post() call

Section 7 — Debugging Tips

Bug 1 — takeUntilDestroyed() called outside injection context

Symptom: Error 'takeUntilDestroyed() can only be used within an injection context' thrown when loadUsers() called from ngOnInit. Cause: takeUntilDestroyed() without argument relies on injection context to grab DestroyRef automatically — only exists during construction. Fix: Inject DestroyRef explicitly (private destroyRef = inject(DestroyRef)) then use takeUntilDestroyed(this.destroyRef) anywhere.

Bug 2 — Response is undefined or missing properties at runtime

Symptom: Cannot read properties of undefined — the response shape doesn't match the interface. Cause: The <T> generic in http.get<T>() is a TypeScript compile-time cast — Angular does not validate at runtime. Fix: Type the wrapper correctly and use map() in the service to extract the inner object. Components should never know about API envelope wrappers.

Bug 3 — CORS error blocking requests in development

Symptom: 'Access to XMLHttpRequest has been blocked by CORS policy'. Cause: Browser (not Angular) blocks cross-origin requests when server doesn't include CORS headers. Fix for development: Use Angular's dev server proxy — proxy.conf.json with /api target pointing to the real API. Angular-specific note: The proxy fix is for development only. In production, CORS must be configured on the server.

Section 8 — Real-World Applications

Scenario 1 — Search with Debounce and Request Cancellation

```
export class SearchComponent {
  private userService = inject(UserService);
  searchControl = new FormControl('');
  results: User[] = [];

  constructor() {
    this.searchControl.valueChanges.pipe(
      debounceTime(300),           // wait 300ms after last keystroke
      distinctUntilChanged(),      // skip if value unchanged
      filter(term => (term?.length ?? 0) >= 2),
      tap(() => this.loading = true),
      switchMap(term =>           // cancel previous request, start new one
        this.userService.searchUsers(term ?? '').pipe(
          catchError(err => { this.error = err.message; return of([]); })
        )
      ),
      takeUntilDestroyed()
    ).subscribe(results => { this.results = results; this.loading = false; });
  }
}
```

Scenario 2 — Parallel API Calls with forkJoin

```
loadDashboard(userId: number): Observable<DashboardData> {
  // forkJoin fires all three simultaneously – completes when ALL three complete
  return forkJoin({
    user:      this.http.get<User>(`/api/users/${userId}`),
    orders:    this.http.get<Order[]>(`/api/users/${userId}/orders`),
    notifications: this.http.get<Notification[]>(`/api/users/${userId}/notifications`)
  }).pipe(
    map(({ user, orders, notifications }) => ({ user, orders, notifications }))
  );
  // NOTE: forkJoin cancels all if ANY observable errors
}
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Create a `PostService` that uses `HttpClient` to call `https://jsonplaceholder.typicode.com/posts`. Define a `Post` interface with `id`, `userId`, `title`, and `body`. Implement `getPosts()` returning `Observable<Post[]>` with `catchError`. In a component, inject `PostService`, display a loading message, show post titles on success, show error on failure. Use `takeUntilDestroyed(destroyRef)`.

Exercise 2 — Intermediate

Extend the `PostService` with `getPostById(id)`, `createPost(post)`, and `deletePost(id)`. Build a component with: list of posts (GET all), Delete button per post (DELETE — optimistically remove from local list), and small create form (POST — prepend new post on success). Handle loading and error states for each operation independently.

Exercise 3 — Advanced

Build a 'user search dashboard' combining three techniques: (1) debounced search using `switchMap`; (2) clicking a user loads detail via `switchMap` (cancels any in-flight request); (3) initial page load uses `forkJoin` to load featured users and categories simultaneously. All three HTTP flows handle errors independently. Use `OnPush` change detection and ensure all subscriptions are properly cleaned up.

Section 10 — Key Takeaways

Core concepts:

- `HttpClient` is an injectable service that returns cold Observables — no request fires until subscription
- Each subscription to an `HttpClient` Observable fires one independent HTTP request
- `http.get<T>()` — the `<T>` is a compile-time type hint, not a runtime validator
- Always put `HttpClient` in services, never in components
- `HttpParams` builds query strings immutably — never concatenate query strings manually
- `catchError` in the service handles HTTP errors before they reach the component
- `takeUntilDestroyed(destroyRef)` cleans up component subscriptions

Task	How
GET with query params	<code>http.get(url, { params: new HttpParams().set('k', v) })</code>
POST with JSON body	<code>http.post(url, body)</code> — Angular sets Content-Type automatically

Task	How
Set custom headers	<code>http.get(url, { headers: new HttpHeaders({ 'X-Custom': 'value' }) })</code>
GET full response	<code>http.get(url, { observe: 'response' }) -> HttpResponse</code>
Parallel requests	<code>forkJoin({ a: obs1, b: obs2 })</code>
Sequential requests	<code>obs1.pipe(switchMap(result => obs2(result)))</code>
Convert to Promise	<code>firstValueFrom(http.get(url))</code>

One-Line Memory Aid: HttpClient returns a cold Observable — nothing happens until you subscribe, and everything that can go wrong should be caught before the component ever sees it.

Section 11 — Thought-Provoking Question + Answer

Question: HttpClient returns Observables, but many developers use firstValueFrom() to convert them to Promises and then use async/await. Is this a legitimate pattern, or an anti-pattern in Angular?

Answer: It's legitimate in specific contexts but trades away Observable superpowers for syntactic convenience. firstValueFrom(http.get<T>(url)) converts the Observable to a Promise. What you lose: Cancellation (Promises can't be cancelled — if component destroys mid-await, the continuation still runs). Composability (you can't pipe debounceTime, switchMap, retry onto a Promise). firstValueFrom is fine in two situations: (1) Route Resolvers that expect a Promise, (2) CanActivate guards where the calling context expects a Promise. It's an anti-pattern when used wholesale as a replacement in components, because it gives up cancellation. Practical rule: Default to Observables with takeUntilDestroyed. Use firstValueFrom only when the calling context genuinely expects a Promise.

Section 12 — Related Topics to Learn Next

- Topic 37 — HTTP Interceptors: centralised auth headers, logging, error handling for all HTTP calls
- Topic 38 — Angular Router: route resolvers use HttpClient to pre-fetch data before navigation
- Topic 39 — Route Guards: guards often call services that use HttpClient to verify permissions
- Topic 46 — RxJS Operators in Depth: switchMap, mergeMap, forkJoin, combineLatest are all essential for advanced HTTP patterns

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 8 (Promises), Topic 13 (Interfaces), Topic 18 (Error Handling), Topic 20 (RxJS), Topic 21 (takeUntilDestroyed), Topic 27 (DI).

Unlocks: Topic 37 (Interceptors), Topic 38 (Router), Topic 39 (Route Guards), Topic 46 (RxJS Operators), Topic 48 (Testing Services & HTTP).

Production readiness: If you can build a typed service with GET/POST/DELETE, handle errors centrally, manage loading and error state, use HttpParams, and clean up with takeUntilDestroyed(destroyRef) — you are production-ready for standard API integration.

Section 14 — Common Interview Questions

Question 1 (Mid) — Why Observable instead of Promise?

'Why does Angular's HttpClient return an Observable instead of a Promise?' Strong answer: Observables are lazy — no request fires until subscription. Observables are cancellable — unsubscribing cancels in-flight request. Observables are composable — pipe(debounceTime, switchMap, retry, catchError). switchMap enables automatic cancellation for searches. takeUntilDestroyed auto-cleans subscriptions. Weak answer misses: The cancellation point — unsubscribing actually cancels the XHR request, not just ignores the response.

Question 2 (Mid-Senior) — Preventing excessive API calls on search

'In a search input calling an API on every keystroke, how do you prevent excessive calls and handle out-of-order responses?' Strong answer: debounceTime(300) — waits 300ms after last keystroke. distinctUntilChanged() — skips if value unchanged. switchMap — cancels previous in-flight request. filter — skips very short queries. Why mergeMap would be wrong — doesn't cancel, causes race conditions. Weak answer misses: switchMap vs mergeMap distinction.

Question 3 (Mid-Senior) — HTTP error handling architecture

'How do you handle HTTP errors in Angular and where should error handling live?' Strong answer: Two layers — catchError in service (translates to domain errors), error callback in component (updates UI). HttpResponse has status, message, error (body), status=0 for network errors. throwError(() => new Error(msg)) re-throws. Service never exposes raw HttpResponse to component. Global errors via HTTP Interceptor. Weak answer misses: The two-layer architecture.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Nested subscriptions

```
// WRONG: nested subscribes for sequential calls - 'Observable hell'
this.userService.getCurrentUser().subscribe(user => {
  this.orderService.getOrders(user.id).subscribe(orders => {
    this.notificationService.getNotifications(user.id).subscribe(notifs => {
      this.data = { user, orders, notifs };
    });
  });
});

// CORRECT: switchMap for sequential, forkJoin for parallel
this.userService.getCurrentUser().pipe(
  switchMap(user =>
    forkJoin({
      orders: this.orderService.getOrders(user.id),
      notifs: this.notificationService.getNotifications(user.id)
    }).pipe(map(rest => ({ user, ...rest })))
  ),
  takeUntilDestroyed(this.destroyRef),
  catchError(err => { this.error = err.message; return EMPTY; })
).subscribe(data => this.data = data);
```

Anti-Pattern 2 — Forgetting provideHttpClient() in app config

```
// WRONG – missing provideHttpClient()
export const appConfig: ApplicationConfig = {
  providers: [provideRouter(routes)] // HttpClient not provided!
};
// Runtime: NullInjectorError: No provider for HttpClient!

// CORRECT
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(withInterceptorsFromDi())
  ]
};
```

Anti-Pattern 3 — Using mergeMap instead of switchMap for user-triggered searches

```
// WRONG: mergeMap keeps ALL in-flight requests – causes race conditions
this.searchControl.valueChanges.pipe(
  debounceTime(300),
  mergeMap(term => this.userService.search(term))
).subscribe(results => this.results = results);

// CORRECT: switchMap cancels previous request when new one arrives
this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.userService.search(term))
).subscribe(results => this.results = results);
```

Section 16 — Mental Model Summary

Core concept in one sentence: HttpClient is an injectable service that wraps every HTTP request as a cold Observable — nothing fires until subscription, everything should be typed, all errors should be caught in the service layer, and all component subscriptions need cleanup.

Key rules:

- 1. HttpClient belongs in services — never inject it directly into components.
- 2. Cold Observable = no request until .subscribe() — the most common beginner trap.
- 3. Always type responses: http.get<User[]>(url) — any destroys type safety.
- 4. catchError in the service translates HTTP errors; error callback in component updates UI.
- 5. Use switchMap for user-triggered searches — cancels stale in-flight requests.
- 6. Use takeUntilDestroyed(destroyRef) for all component HTTP subscriptions.

The single most important Angular rule: Put HttpClient in a service, type every response with an interface, catch errors with catchError, and clean up with takeUntilDestroyed(destroyRef) — every omission is a production bug waiting to happen.

Scenario	Operator	Why
Sequential calls (A feeds B)	switchMap	Chains and cancels previous
Parallel independent calls	forkJoin	Fires all at once, waits for all
User search (cancel previous)	switchMap	Cancels in-flight stale requests
Retry on failure	retry(3)	Re-subscribes up to N times

Scenario	Operator	Why
Upload queue (preserve order)	concatMap	Queues sequentially

TOPIC 37 — HTTP Interceptors

Angular Core — HTTP & Routing | Difficulty: Intermediate | Angular Relevance: Critical

Section 0 — Difficulty & Priority Rating

Difficulty: Intermediate — The concept is straightforward once you understand middleware pipelines, but the Angular-specific setup (functional vs class-based, `provideHttpClient` configuration, cloning requests) has enough nuance to trip up developers who learn the pattern without understanding the immutability requirement.

Angular Relevance: Critical — Interceptors are the production solution for cross-cutting HTTP concerns: auth tokens, logging, error handling, loading indicators, and request retries. Without them, these concerns get duplicated across every service.

Section 1 — Explanation

The Airport Security Analogy

Imagine every HTTP request your app makes is a passenger trying to board a flight, and every response is a passenger returning from a trip.

Without an interceptor: Each passenger handles their own security check, their own customs declaration, their own baggage tagging — separately, every single time.

With an interceptor: You install a security checkpoint that every passenger passes through automatically. The checkpoint can: add something to every outbound bag (attach an auth token), inspect every inbound bag (log every response), stop a bag and reroute it (retry on 401, redirect to login on 403), count bags passing through (show a global loading spinner). The passengers (your services) don't know the checkpoint exists.

This is the Airport Security Analogy — interceptors are checkpoints in the HTTP pipeline that every request and response traverses, invisibly, automatically.

The chain works like this: `HttpClient` call -> Interceptor 1 (outbound) -> Interceptor 2 (outbound) -> Network -> Interceptor 2 (inbound) -> Interceptor 1 (inbound) -> Subscriber.

	Functional Interceptor (modern)	Class-based Interceptor (legacy)
Syntax	<code>HttpInterceptorFn</code> — a plain function	implements <code>HttpInterceptor</code> class
Registration	<code>provideHttpClient(withInterceptors([...]))</code>	<code>HTTP_INTERCEPTORS</code> multi-provider token
Tree-shaking	Yes	No
DI inside interceptor	<code>inject()</code> at call time	Constructor injection
Angular version	v15+ preferred	All versions

The immutability rule: `HttpRequest` objects are immutable. To change a request, you must call `request.clone({ ... })` to get a new request, then pass the cloned version to next.

Section 2 — Connection to Prior Concepts

Directly builds on:

- Topic 36 — `HttpClient`: interceptors are a layer on top of `HttpClient`; you need to understand the base flow first

- Topic 20 — RxJS Subscription Management: interceptors return Observables and can pipe RxJS operators onto the response stream
- Topic 27 — Services & DI: interceptors use inject() to access services (auth tokens, loading state, router)
- Topic 18 — Error Handling: catchError inside interceptors is how global HTTP error handling works
- Topic 23 — Angular Signals: a loading signal in a service can be toggled by an interceptor to drive a global spinner

Extends most directly: Topic 36 — HttpClient, because interceptors are the cross-cutting infrastructure layer that completes the HTTP architecture.

Section 3 — Code Example

Plain concept — the interceptor contract

```
// The functional interceptor signature – a function matching HttpInterceptorFn
const myInterceptor: HttpInterceptorFn = (
  req: HttpRequest<unknown>,
  next: HttpHandlerFn
): Observable<HttpEvent<unknown>> => {

  // 1. Inspect or modify the request (must clone – requests are immutable)
  const modifiedReq = req.clone({
    headers: req.headers.set('X-Custom-Header', 'hello')
  });

  // 2. Pass to next interceptor (or network) and return the response Observable
  return next(modifiedReq);

  // 3. Optionally pipe operators onto the response Observable
  // return next(modifiedReq).pipe(tap(event => console.log('Response:', event)));
};
```

Auth Interceptor

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token       = authService.getToken();

  if (!token) return next(req); // skip if no token

  // Clone the request – HttpRequest is immutable, never mutate directly
  const authenticatedReq = req.clone({
    headers: req.headers.set('Authorization', `Bearer ${token}`)
  });

  return next(authenticatedReq);
};
```

Error Interceptor

```
export const errorHandler: HttpInterceptorFn = (req, next) => {
  const router = inject(Router);
  const toastService = inject(ToastService);

  return next(req).pipe(
    catchError((error: HttpResponse) => {
      if (error.status === 401) {
        router.navigate(['/login'], { queryParams: { returnUrl: router.url } });
      }
      if (error.status >= 500) {
        toastService.show('Server error – please try again later', 'error');
      }
      return throwError(() => error); // re-throw so services can also handle it
    })
  );
};
```

Loading Interceptor with Signal-based counter

```
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);
  loadingService.increment();

  return next(req).pipe(
    finalize(() => loadingService.decrement()) // runs on complete AND error
  );
};

@Injectable({ providedIn: 'root' })
export class LoadingService {
  private requestCount = signal(0);
  isLoading = computed(() => this.requestCount() > 0);
  increment() { this.requestCount.update(n => n + 1); }
  decrement() { this.requestCount.update(n => Math.max(0, n - 1)); }
}
```

Logging Interceptor

```
export const loggingInterceptor: HttpInterceptorFn = (req, next) => {
  const startTime = Date.now();

  return next(req).pipe(
    tap({
      next: event => {
        if (event instanceof HttpResponse) {
          const elapsed = Date.now() - startTime;
          console.log(`[HTTP] ${req.method} ${req.url} -> ${event.status} (${elapsed}ms)`);
        }
      },
      error: error => {
        console.error(`[HTTP ERROR] ${req.method} ${req.url} -> ${error.status}`);
      }
    })
  );
};
```

Registering interceptors in app.config.ts

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    provideHttpClient(
      withInterceptors([
        // Order matters: top-to-bottom on request, bottom-to-top on response
        loggingInterceptor, // outermost – sees everything
        loadingInterceptor,
        authInterceptor,    // attaches token before sending
        errorInterceptor,   // innermost – closest to actual response
      ])
    )
  ]
};
```

Section 4 — Before & After Refactor

Before — Auth token added manually in every service

```
// WRONG: auth header duplicated in every service method
@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers() {
    const token = localStorage.getItem('auth_token');
    const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
    return this.http.get<User[]>('/api/users', { headers }); // duplicated
  }
  deleteUser(id: number) {
    const token = localStorage.getItem('auth_token');
    const headers = new HttpHeaders({ Authorization: `Bearer ${token}` });
    return this.http.delete(`/api/users/${id}`, { headers }); // duplicated again
  }
}
```

After — Auth handled once in an interceptor

```
// CORRECT: Services are clean – zero auth logic
@Injectable({ providedIn: 'root' })
export class UserService {
  getUsers() { return this.http.get<User[]>('/api/users'); }
  deleteUser(id: number) { return this.http.delete(`/api/users/${id}`); }
}
// Token attached by authInterceptor – invisible to every service
```

Section 5 — Common Mistakes & Misconceptions

Mistake 1 — Mutating the request instead of cloning it

What: `req.headers.set('Authorization', 'Bearer token')` — `HttpHeaders` is immutable, `.set()` returns a new Headers object, not applied. Why: `req.headers` looks like a mutable object. How to avoid: Always use `req.clone({ headers: req.headers.set(...) })`. Angular-specific consequence: Header is silently not added. The API returns 401. Developer debugs the auth service when the bug is in the interceptor.

Mistake 2 — Intercepting requests that should be excluded

What: Sending auth token to third-party CDNs or public APIs. Why: The interceptor is meant to be universal. How to avoid: Add a URL check — only attach auth headers to your own API: if

(!req.url.startsWith('https://api.myapp.com')) return next(req). Angular-specific consequence: Your private auth token is leaked to third-party servers — a security vulnerability.

Mistake 3 — Registering interceptors in the wrong order

What: Registering auth interceptor after error interceptor expecting it to run first. Why: Order isn't obvious. How to avoid: Interceptors run top-to-bottom on outbound requests and bottom-to-top on inbound responses. The first interceptor in the array is the outermost wrapper.

Angular-specific consequence: Auth tokens not attached before error handling runs, or logging interceptor sees request without token.

Section 6 — Do's and Don'ts

DO	DON'T
Always <code>req.clone({...})</code> to modify a request	Call <code>req.headers.set(...)</code> and pass original req to next
Check request URL before attaching auth tokens	Send auth headers to every URL including third-party services
Use <code>finalize()</code> for cleanup — runs on both complete and error	Use <code>tap</code> for cleanup — <code>tap</code> 's complete callback doesn't run on error
Use <code>inject()</code> inside the interceptor function body	Try to add constructor injection to functional interceptors
Register in <code>provideHttpClient(withInterceptors([...]))</code>	Register functional interceptors using legacy <code>HTTP_INTERCEPTORS</code> token
Handle errors and re-throw with <code>throwError()</code>	Swallow errors without re-throwing — breaks per-service error handling

Section 7 — Debugging Tips

Bug 1 — Interceptor not running at all

Symptom: Auth headers missing, loading spinner never shows, interceptor code never reached.

Cause: (a) `withInterceptors([...])` missing from `provideHttpClient()`; (b) class reference passed instead of function reference; (c) legacy `NgModule` setup missing `HTTP_INTERCEPTORS` provider. Fix: Add `console.log` as first line of interceptor. If it never logs, registration is the problem.

Bug 2 — Infinite redirect loop on 401

Symptom: A 401 response triggers `router.navigate(['/login'])`, but the login page itself makes an HTTP call returning 401, causing another redirect. Cause: Error interceptor catches 401 on every request, including requests from the login page itself. Fix: Check `const isLoginRequest = req.url.includes('/auth/login')` and `const isLoginPage = router.url.includes('/login')` before redirecting.

Bug 3 — Loading spinner never hides after an HTTP error

Symptom: Loading spinner appears when request starts but stays permanently after a failed request. Cause: Loading interceptor uses `tap` and relies on complete callback to decrement counter. But when an `Observable` errors, it does not call `complete` — it calls `error`. Fix: Use `finalize(() => loadingService.decrement())` — `finalize` runs on both `complete` AND `error`. This is the most common loading-interceptor bug.

Section 8 — Real-World Applications

Scenario 1 — Token Refresh Interceptor

```
export const tokenRefreshInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);

  return next(req).pipe(
    catchError((error: HttpResponse) => {
      const isRefreshRequest = req.url.includes('/auth/refresh');
      if (error.status !== 401 || isRefreshRequest) {
        return throwError(() => error);
      }

      return authService.refreshToken().pipe(
        switchMap(newToken => {
          const retryReq = req.clone({
            headers: req.headers.set('Authorization', `Bearer ${newToken}`)
          });
          return next(retryReq);
        }),
        catchError(refreshError => {
          authService.logout();
          return throwError(() => refreshError);
        })
      );
    })
  );
};
```

Scenario 2 — Environment-Aware Base URL Interceptor

```
export const baseUrlInterceptor: HttpInterceptorFn = (req, next) => {
  const envService = inject(EnvironmentService);

  // Only modify relative URLs – leave absolute URLs (third-party) untouched
  if (req.url.startsWith('http')) return next(req);

  const apiReq = req.clone({
    url: `${envService.apiBaseUrl}${req.url}`
    // e.g. '/api/users' -> 'https://api.myapp.com/api/users'
  });

  return next(apiReq);
};

// Services stay clean, no environment awareness needed:
getUsers() { return this.http.get<User[]>('/api/users'); }
```

Section 9 — Practice Exercises

Exercise 1 — Beginner

Write a functional interceptor called `contentTypeInterceptor` that adds a `Content-Type: application/json` header to every outbound POST and PUT request, but leaves GET and DELETE requests unchanged. Register it in `app.config.ts`. Verify by logging `req.headers.get('Content-Type')` inside the interceptor.

Exercise 2 — Intermediate

Build a `retryInterceptor` that automatically retries any failed request up to 2 times if the error status is 503 (Service Unavailable), with a 1-second delay between retries. Requests that fail for

any other reason should not be retried — they should error immediately. Use the RxJS retry operator with a delay configuration. Log each retry attempt to the console.

Exercise 3 — Advanced

Build a complete auth interceptor pipeline consisting of three interceptors: (1) `authInterceptor` — attaches Bearer token to all requests matching `https://api.myapp.com/*`; (2) `tokenRefreshInterceptor` — on 401, calls `authService.refreshToken()`, then retries original request with new token (with loop prevention for the refresh endpoint); (3) `sessionExpiredInterceptor` — if refresh also fails with 401, calls `authService.logout()` and navigates to `/session-expired`. Register all three in the correct order and explain why the order matters.

Section 10 — Key Takeaways

Core concepts:

- An interceptor is a function sitting between `HttpClient` and the network, running on every request and response
- Functional interceptors match `HttpInterceptorFn` — they receive `(req, next)` and return `Observable<HttpEvent<unknown>>`
- `HttpRequest` is immutable — always `req.clone({...})` to modify a request, never mutate directly
- Interceptors ordered: top-to-bottom on outbound requests, bottom-to-top on inbound responses
- `inject()` works inside functional interceptors — call at top of function body
- `finalize()` is the correct operator for cleanup — runs on both complete and error; `tap(complete)` does not

Task	How
Add a request header	<code>req.clone({ headers: req.headers.set('key', 'value') })</code>
Replace the URL	<code>req.clone({ url: newUrl })</code>
Add query params	<code>req.clone({ params: req.params.set('key', 'value') })</code>
Inspect the response	<code>next(req).pipe(tap(event => ...))</code>
Handle errors globally	<code>next(req).pipe(catchError(err => ...))</code>
Run cleanup always	<code>next(req).pipe(finalize(() => ...))</code>
Skip interceptor for a URL	<code>if (req.url.includes('skip')) return next(req)</code>

One-Line Memory Aid: An interceptor is invisible middleware for every HTTP call — clone the request to modify it, pipe operators to handle the response, and register in order because sequence is execution order.

Section 11 — Thought-Provoking Question + Answer

Question: If you have both a per-service `catchError` (in `UserService`) and a global `errorInterceptor`, which one runs first on a 500 response — and what happens if the interceptor swallows the error instead of re-throwing it?

Answer: The interceptor runs first. The response travels back through the interceptor chain before it reaches the service's `pipe(catchError(...))`. If the interceptor swallows the error — calling

catchError but returning of(null) or EMPTY instead of throwError() — the error is consumed. The Observable switches to a success path emitting null or completing immediately. The service's catchError never runs. The component's error callback never runs. The component receives null as its response data — and if it tries to access null.users, it throws a different error. The original 500 is invisible. Practical rule: An interceptor should either re-throw as-is (throwError(() => error)), re-throw as a new error (throwError(() => new Error(msg))), or only return a non-error Observable when you are certain the component can proceed without the real data.

Section 12 — Related Topics to Learn Next

- Topic 38 — Angular Router: interceptors often call the Router for redirects on 401/403
- Topic 39 — Route Guards: guards and interceptors often share auth service logic
- Topic 46 — RxJS Operators in Depth: switchMap, catchError, retry, finalize used heavily in interceptors
- HttpContext: Angular's mechanism for passing metadata into an interceptor from the call site

Section 13 — Study Plan Checkpoint

Directly depends on: Topic 18 (Error Handling), Topic 20 (RxJS), Topic 27 (DI), Topic 36 (HttpClient).

Unlocks: Topic 38 (Router — interceptors navigate programmatically), Topic 39 (Route Guards — share auth service with interceptors), Topic 46 (RxJS Operators — token refresh, retry patterns), Topic 48 (Testing Services & HTTP).

Production readiness: If you can write an auth interceptor that clones the request, attaches a token for your own API only, handles 401 with a redirect, uses finalize for loading state, and is registered in the correct order — you are production-ready. The token refresh pattern is the advanced tier — aim for it once the fundamentals are solid.

Section 14 — Common Interview Questions

Question 1 (Mid) — What is an HTTP interceptor?

'What is an HTTP interceptor in Angular and what are the most common use cases?' Strong answer: Middleware that intercepts every HTTP request/response transparently. Common uses: attaching auth tokens, global error handling (401 redirect), loading state management, logging, base URL injection. Modern Angular uses functional interceptors registered via withInterceptors(). Requests are immutable — must use req.clone(). Weak answer misses: The immutability requirement.

Question 2 (Mid) — Global loading spinner

'How would you implement a global loading spinner that activates during any HTTP request?' Strong answer: LoadingService with signal tracking in-flight request count. Loading interceptor increments on start and decrements in finalize(). Why finalize — it runs on both complete and error; tap(complete) does not. Counter approach handles concurrent requests correctly. Spinner component subscribes to loadingService.isLoading computed signal. Weak answer misses: Counter vs boolean — boolean breaks with concurrent requests.

Question 3 (Senior) — Silent token refresh

'Walk me through implementing silent token refresh — automatically refreshing an expired JWT and retrying the original request.' Strong answer: Intercept 401 in error interceptor. Call `authService.refreshToken()`. Use `switchMap` to retry original request with new token via `req.clone()`. Loop prevention — check `req.url` includes `/auth/refresh`. Concurrent refresh race condition — use a shared Observable with `shareReplay(1)` so only one refresh call is made. Fall back to `authService.logout()` if refresh fails. Weak answer misses: The concurrent refresh race condition.

Section 15 — Anti-Patterns to Avoid in Production

Anti-Pattern 1 — Boolean instead of counter for loading state

```
// WRONG: boolean breaks with concurrent requests
// Request A starts -> isLoading = true
// Request B starts -> isLoading = true (no change)
// Request A finishes-> isLoading = false <-- spinner hides while B still running!
loadingService.setLoading(true);
return next(req).pipe(finally(() => loadingService.setLoading(false)));

// CORRECT: counter – spinner stays until ALL requests are done
loadingService.increment();
return next(req).pipe(finally(() => loadingService.decrement()));

// LoadingService:
isLoading = computed(() => this.requestCount() > 0);
```

Anti-Pattern 2 — Swallowing errors in the interceptor

```
// WRONG: catchError returns EMPTY – error silently consumed
return next(req).pipe(
  catchError((error: HttpResponse) => {
    console.error('HTTP error:', error.status);
    return EMPTY; // Observable completes without emitting – all downstream handlers dead
  })
);

// CORRECT: always re-throw
return next(req).pipe(
  catchError((error: HttpResponse) => {
    if (error.status === 401) router.navigate(['/login']);
    return throwError(() => error); // re-throw – services and components still handle it
  })
);
```

Anti-Pattern 3 — Expensive synchronous work in the interceptor for every request


```
// WRONG: full JWT decode on every single HTTP request
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).getToken();
  const decoded = JSON.parse(atob(token.split('.')[1])); // CPU work on every request
  const isExpired = decoded.exp < Date.now() / 1000;
  // ...
};

// CORRECT: delegate expiry logic to AuthService – cached, not recomputed per request
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const token = authService.getToken(); // AuthService caches expiry internally
  if (!token) return next(req);
  return next(req.clone({
    headers: req.headers.set('Authorization', `Bearer ${token}`)
  }));
};
```

Section 16 — Mental Model Summary

Core concept in one sentence: An HTTP interceptor is invisible middleware that every request and response passes through automatically — used to attach auth tokens, handle errors, track loading state, and log requests without touching a single service or component.

Key rules:

- 1. HttpRequest is immutable — always `req.clone({...})` to modify; never mutate `req` directly.
- 2. Interceptors ordered: top-to-bottom on requests, bottom-to-top on responses.
- 3. Use `inject()` inside the interceptor function body to access Angular services.
- 4. Use `finalize()` for cleanup — it runs on both complete and error; `tap(complete)` does not.
- 5. Always re-throw errors with `throwError()` unless certain the component can handle null data.
- 6. Exclude your auth/refresh endpoint from 401 redirect logic to prevent infinite loops.

The single most important Angular rule: Always `req.clone()` — never mutate the request — and always `throwError()` after handling an error — never swallow it silently.

Interceptor order cheat sheet:

```
withInterceptors([
  loggingInterceptor,    <- runs 1st on request, 4th on response
  loadingInterceptor,    <- runs 2nd on request, 3rd on response
  baseUrlInterceptor,    <- runs 3rd on request, 2nd on response
  authInterceptor,       <- runs 4th on request, 1st on response (closest to network)
])
```

Cross-cutting concern	Correct tool
Attach auth token	<code>authInterceptor</code> — clone + set header
Global error handling (401 redirect)	<code>errorInterceptor</code> — <code>catchError</code> + <code>throwError</code>
Loading spinner	<code>loadingInterceptor</code> — counter + <code>finalize</code>
Request/response logging	<code>loggingInterceptor</code> — <code>tap</code> + <code>Date.now()</code>
Token refresh on 401	<code>tokenRefreshInterceptor</code> — <code>catchError</code> + <code>switchMap</code>

Cross-cutting concern	Correct tool
Prepend environment base URL	baseUrlInterceptor — check relative URL + clone
